

5. 토큰화

단어 및 글자 토큰화

단어 토큰화

단어 토큰화의 한계 → OOV 문제

글자 토큰화

한국어 글자 토큰화 — 자소 단위

글자 토큰화의 장점과 한계

형태소 토큰화

형태소 어휘 사전

품사 태깅 (POS Tagging, Part Of Speech Tagging)

KoNLPy

Okt (Open Korean Text)

꼬꼬마 (Kkma)

Okt vs. 꼬꼬마 비교

NLTK (Natural Language Toolkit)

주요 기능

KoNLPy와의 차이

spaCy

사용 방식 — 객체 지향

NLTK vs. spaCy 비교

형태소 분석 & 품사 태깅의 한계와 의의

하위 단어 토큰화 (Subword Tokenization)

바이트 페어 인코딩 (BPE, Byte Pair Encoding)

작동 방식

말뭉치 적용 예시

BPE의 핵심 장점 — OOV 해결

센텐스피스 (Sentencepiece)

코포라 (Korpora)

설치

토큰나이저 모델 학습

워드피스 (Wordpiece)

BPE와의 차이

토큰나이저스 (Tokenizers)

주요 기능

예시 코드

자연어란?

- 프로그래밍 언어처럼 인공적으로 설계된 언어가 아니라, 사람이 일상에서 자연스럽게 사용하는 언어를 말한다. **자연어 처리(NLP)**는 컴퓨터가 이 자연어를 이해하고 해석·생성할 수 있게 하는 기술로, 인공지능의 하위 분야다.

NLP가 어려운 이유 — 3가지 핵심 문제

- 자연어는 프로그래밍 언어와 달리 규칙이 불완전하고 맥락 의존적이다. 그래서 NLP 모델이 해결해야 할 문제가 크게 세 가지 있다.
 - **모호성 (Ambiguity):** 같은 단어나 구가 문맥에 따라 전혀 다른 의미를 가질 수 있다. 알고리즘은 이 다양한 의미를 구분해야 한다.
 - **가변성 (Variability):** 사투리, 강세, 신조어, 작문 스타일 등으로 언어 표현이 매우 다양하다. 같은 의미라도 표현 방식이 천차만별이다.

- **구조 (Structure):** 문장의 의미를 파악하려면 구문(Syntactic) 구조를 먼저 파악하고 거기서 의미(Semantic)를 추론해야 한다.

이 문제들을 다루기 위해 NLP 모델은 대량의 언어 데이터로 학습된다. 여기서 **말뭉치(Corpus)** 개념이 등장한다.

말뭉치 (Corpus) — 학습 데이터

- 말뭉치란 NLP 모델을 훈련하고 평가하는 데 쓰이는 대규모 자연어 텍스트 데이터다. 뉴스 기사, 사용자 리뷰, 저널, 칼럼 등 목적에 따라 다양하게 구축된다.
- 그런데 컴퓨터는 텍스트를 그대로 처리하지 못한다. 먼저 텍스트를 더 작은 단위로 쪼개야 하는데, 이 단위가 **토큰(Token)** 이다.

단어 및 글자 토큰화

토큰화란?

텍스트 데이터를 구조적으로 분해해 개별 토큰으로 나누는 전처리 과정. 단어/문장의 빈도수, 출현 패턴 파악이 가능해지고, 기계 번역·문서 분류·감성 분석 등 다양한 NLP 작업의 기반이 된다.

텍스트를 나누는 단위에 따라 **단어 토큰화**와 **글자 토큰화** 두 가지로 구분된다.

단어 토큰화

텍스트를 **의미 있는 단위인 단어로** 분리하는 작업.

띄어쓰기, 문장 부호, 대소문자 등 특정 구분자를 기준으로 토큰화가 수행된다. 품사 태깅, 개체명 인식, 기계 번역 등에서 가장 널리 쓰이는 방법이다.

Python에서는 `split()` 메서드로 간단히 구현할 수 있다. 구분자를 지정하지 않으면 **공백(Whitespace)** 기준으로 나눈다.

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
tokenized = review.split()
# ['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거움은', '최고!', ...]
```

단어 토큰화의 한계 → OOV 문제

단어 토큰화는 **표면 형태**를 기준으로 나누기 때문에, 같은 의미라도 형태가 조금만 달라지면 서로 다른 토큰으로 취급된다.

예를 들어 `'최고!'` 와 `'최고!!'` 는 느낌표 수만 다르지만 다른 토큰이 되고, `'cg'` 와 `'cg.'` 도 마찬가지다. 단어 사전에 `'cg'` 가 있어도 `'cg.'`, `'cg는'`, `'cg도'` 는 모두 **OOV**가 된다.

이처럼 단어 토큰화는 **한국어 접사, 문장 부호, 오타, 띄어쓰기 오류** 등에 취약하다는 근본적인 한계가 있다.

글자 토큰화

띄어쓰기뿐 아니라 **글자(문자)** 단위로 문장을 나누는 방식. 단어 토큰화보다 훨씬 작은 단어 사전을 구축할 수 있어 학습 시 컴퓨터 자원을 아낄 수 있고, 각 단어를 더 자주 학습할 수 있다는 장점이 있다.

Python에서는 `list()` 로 간단히 구현한다.

```
tokenized = list(review)
```

```
# ['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ...]
```

단어 토큰화와 달리 **공백도 토큰**으로 나뉜다.

한국어 글자 토큰화 — 자소 단위

영어는 글자 토큰화 시 알파벳 단위로 나뉘면 되지만, **한국어는 한 글자가 초성·중성·종성(자소)의 조합**으로 이루어져 있다. 따라서 한국어는 글자보다 더 세밀한 **자소 단위** 토큰화가 필요하다.

이를 위해 `jamo` 라이브러리를 사용한다. `jamo`는 한글 음절 분해 및 합성 라이브러리로, 두 가지 핵심 함수를 제공한다.

- `h2j(hangul_string)` — 완성형 한글을 **조합형 자모**로 변환
- `j2hcj(jamo)` — 조합형 자모 문자열을 **자소 단위**로 분리해 반환

완성형: 조합된 글자 자체에 값을 부여해 인코딩 (예: '한' → 하나의 코드)

조합형: 자모를 따로 인코딩한 뒤 조합해 한글 표현

```
from jamo import h2j, j2hcj
decomposed = j2hcj(h2j(review))
tokenized = list(decomposed)
# ['ㅎ', 'ㄷ', 'ㄴ', 'ㅅ', 'ㅣ', 'ㄹ', 'ㄱ', '과', ' ', ...]
```

글자 토큰화의 장점과 한계

- **장점** — 단어 토큰화의 OOV 문제를 크게 줄일 수 있다. `'cg도'`, `'cg는'`, `'cg.'` 같은 경우도 자소 단위로 쪼개면 `'ㄷ', 'ㅇ', 'ㄴ', 'ㅡ', 'ㄴ'` 등 이미 사전에 있는 자소들로 처리된다. 작은 사전으로도 OOV를 획기적으로 줄일 수 있다.
- **한계** — 개별 토큰 자체는 아무런 의미가 없기 때문에, 모델이 각 토큰을 조합해 의미를 직접 도출해야 한다. 또한 토큰 수가 크게 늘어나 **입력 시퀀스 길이가 급격히 증가**한다.

같은 문장 기준 시퀀스 길이 비교:

- 단어 토큰화: **13**
- 글자 토큰화: **53**
- 자소 단위 토큰화: **101**

시퀀스가 길어질수록 모델의 연산량이 증가한다는 것이 글자 토큰화의 근본적인 trade-off 이다.

형태소 토큰화

텍스트를 형태소 단위로 나누는 토큰화 방법. 단순히 띄어쓰기나 글자로 나누는 게 아니라, 언어의 문법과 구조를 고려해 의미 있는 최소 단위로 분리한다.

왜 한국어에서 특히 중요한가?

- 한국어는 교착어(Agglutinative Language)로, 어근에 다양한 접사·조사가 붙어 하나의 낱말을 이룬다. 띄어쓰기만으로는 의미 단위를 제대로 구분하기 어렵다.
- 예: `'그는 나에게 인사를 했다'`
 - `'그는'` = `'그'`(어근) + `'는'`(조사)

- '나에게' = '나' (어근) + '에게' (조사)
- 단어 토큰화로 나누면 '그는' 과 '그가' 는 다른 토큰이 되지만, 형태소로 나누면 둘 다 '그' 라는 같은 어근을 공유한다는 것을 파악할 수 있다.

형태소의 두 종류

- **자립 형태소 (Free Morpheme)** — 스스로 의미를 가지는 형태소. 명사, 동사, 형용사 등 단어의 기본 단위.
 - 예: **그**, **나**, **인사**
- **의존 형태소 (Bound Morpheme)** — 단독으로는 의미를 갖지 못하고, 자립 형태소와 결합해 문장에서 특정 역할을 수행. 조사, 어미, 접두사, 접미사 등.
 - 예: **-는**, **-에게**, **-를**, **했-**, **-다**

형태소 어휘 사전

일반 어휘 사전에서 각 단어의 **형태소 정보까지 포함한** 사전. 단어가 어떤 형태소들의 조합으로 이루어져 있는지를 담고 있어, 형태소 분석 작업의 핵심 자원이다.

왜 필요한가? — 일반 어휘 사전의 한계

- 띄어쓰기 기반 단어 토큰화로 어휘 사전을 만들면, '그는', '나는', '그녀는' 이 모두 **별개의 토큰**으로 등록된다. 모델은 이것들이 같은 의미 구조(어근 + 조사 '-는')를 공유한다는 걸 알지 못한 채 학습한다.
- 형태소 단위로 사전을 구축하면 '그', '나', '그녀' + '-는' 으로 분리되므로, 새로운 단어 '그녀는' 이 등장해도 '그녀' 토큰 하나만 새로 학습하면 된다. 이는 **OOV 문제를 줄이고 사전 크기도 효율적으로 유지**할 수 있게 해준다.
- 또한 형태소 어휘 사전에는 각 형태소가 **어떤 품사에 속하는지, 그 뜻이 무엇인지도** 함께 제공된다.

품사 태깅 (POS Tagging, Part Of Speech Tagging)

형태소 분석 후 각 형태소에 해당하는 **품사를 태깅**하는 작업. 품사 정보를 통해 모델이 문맥을 더 정확히 파악할 수 있다.

예: "그는 나에게 인사를 했다"

그(명사) + 는(조사) + 나(명사) + 에게(조사) + 인사(명사) + 를(조사) + 했다(동사)

실습에서는 **KoNLPy**, **NLTK**, **spaCy** 라이브러리를 활용해 형태소 단위 토큰화와 품사 태깅을 수행한다.

KoNLPy

한국어 자연어 처리를 위한 Python 라이브러리. 명사 추출, 형태소 분석, 품사 태깅 등을 제공하며 텍스트 마이닝, 감성 분석, 토픽 모델링 등 다양한 NLP 작업에 사용된다.

- 설치: `pip install konlpy`
- **Java 기반(JDK 필요)** — 실행 전 JDK가 설치되어 있어야 한다.

지원하는 형태소 분석기: Okt, 꼬꼬마(Kkma), 코모란(Komorán), 한나눔(Hannanum), 메캅(Mecab)

Okt (Open Korean Text)

SNS 텍스트 데이터 기반으로 개발된 분석기. 4가지 주요 메서드를 제공한다.

메서드	기능	반환 형태
<code>okt.nouns()</code>	명사 추출	리스트
<code>okt.phrases()</code>	어절 구 단위 추출	리스트
<code>okt.morphs()</code>	형태소 추출	리스트
<code>okt.pos()</code>	품사 태깅	(형태소, 품사) 튜플 리스트

```
# Okt 토큰화
from konlpy.tag import Okt

okt = Okt()

sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."

nouns = okt.nouns(sentence)
phrases = okt.phrases(sentence)
morphs = okt.morphs(sentence)
pos = okt.pos(sentence)

print("명사 추출 :", nouns)
print("구 추출 :", phrases)
print("형태소 추출 :", morphs)
print("품사 태깅 :", pos)
```

▼ 실행 결과

```
명사 추출 : ['무엇', '상상', '수', '사람', '무엇', '날', '수']
구 추출 : ['무엇', '상상', '상상할 수', '상상할 수 있는 사람', '사람']
형태소 추출 : ['무엇', '이든', '상상', '할', '수', '있', '는', '사', '람', '은', '무엇', '이든', '만', '들', '어', '낼', '수', '있', '다', '.']
품사 태깅 : [(('무엇', 'Noun'), ('이든', 'Josa'), ('상상', 'Noun'), ('할', 'Verb'), ('수', 'Noun'), ('있', 'Adjective'), ('는', 'Particle'), ('사', 'Noun'), ('람', 'Noun'), ('은', 'Josa'), ('무엇', 'Noun'), ('이든', 'Josa'), ('만', 'Verb'), ('들', 'Noun'), ('어', 'Noun'), ('낼', 'Verb'), ('수', 'Noun'), ('있', 'Adjective'), ('다', 'Verb'), ('.', 'Punctuation'))]
```

표 5.1 Okt 품사 태그

태그	품사	태그	품사
Noun	명사	Punctuation	구두점
Verb	동사	Foreign	외국어
Adjective	형용사	Alpha	알파벳
Determiner	관형사	Number	숫자
Adverb	부사	KoreanParticle	자음/모음
Conjunction	접속사	URL	웹 주소
Exclamation	감탄사	Hashtag	해시태그(#)
Josa	조사	Email	이메일
PreEomi	선어말어미	ScreenName	아이디 표기(@)
Eomi	어미	Unknown	미등록
Suffix	접미사		

꼬꼬마 (Kkma)

국립국어원 세종 말뭉치 기반으로 학습된 분석기. Okt와 메서드 구성은 비슷하지만 `phrases()` 대신 `sentences()` (문장 추출)를 제공한다.

```
# 꼬꼬마 토큰화
from konlpy.tag import Kkma

kkma = Kkma()

sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."

nouns = kkma.nouns(sentence)
sentences = kkma.sentences(sentence)
morphs = kkma.morphs(sentence)
pos = kkma.pos(sentence)

print("명사 추출 :", nouns)
print("문장 추출 :", sentences)
print("형태소 추출 :", morphs)
print("품사 태깅 :", pos)
```

▼ 실행 결과

```
명사 추출 : ['무엇', '상상', '수', '사람', '무엇']
문장 추출 : ['무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다.']
형태소 추출 : ['무엇', '이', '든', '상상', '하', 'ㄹ', '수', '있', '는', '사람', '은', '무엇', '이', '든', '만들', '어',
품사 태깅 : [('무엇', 'NNG'), ('이', 'VCP'), ('든', 'ECE'), ('상상', 'NNG'), ('하', 'XSV'), ('ㄹ', 'ETD'), ('수',
```

표 5.2 꼬꼬마 품사 태그

태그	품사	태그	품사
NNG	보통명사	EPH	존칭 선어말 어미
NNP	고유명사	EPT	시제 선어말 어미
NNB	일반 의존 명사	EPP	공손 선어말 어미
NNM	단위 의존 명사	EFN	평서형 종결 어미
NR	수사	EFQ	의문형 종결 어미
NP	대명사	EFO	명령형 종결 어미
VV	동사	EFA	청유형 종결 어미
VA	형용사	EFI	감탄형 종결 어미
VXV	보조 동사	EFR	존칭형 종결 어미
VXA	보조 형용사	ECE	대등 연결 어미
VCP	긍정 지정사	ECS	보조적 연결 어미
VCN	부정 지정사	ECD	의존적 연결 어미
MDN	수 관형사	ETN	명사형 전성 어미
MDT	일반 관형사	ETD	관형형 전성 어미

태그	품사	태그	품사
MAG	일반 부사	XPV	체언 접두사
MAC	접속 부사	XPV	용언 접두사
JKS	주격 조사	XSN	명사파생 접미사
JKC	보격 조사	XSV	동사 파생 접미사
JKG	관형격 조사	XSA	형용사 파생 접미사
JKO	목적격 조사	XR	어근
JKM	부사격 조사	SF	마침표, 물음표, 느낌표
JKI	호격 조사	SE	줄임표
JKQ	인용격 조사	SS	따옴표, 괄호표, 줄표
JC	접속 조사	SP	쉼표, 기온뎡점, 콜론, 빗금
JX	보조사	SO	붙임표(물결, 숨김, 빼짐)
OH	한자	SW	기타기호(논리수학기호, 화폐기호)
OL	외국어	IC	감탄사
ON	숫자	UN	명사추정범주

Okt vs. 꼬꼬마 비교

동일한 문장에도 분석기마다 결과가 다르다.

- **명사 추출:** Okt는 '넌', '수' 를 명사로 반환하지만 꼬꼬마는 반환하지 않음
- **형태소 추출:** 꼬꼬마는 '할' 을 '하' + 'ㄹ' 로 더 세밀하게 분리
- **품사 태깅:** Okt 19개 품사 vs 꼬꼬마 56개 품사. 세분화할수록 정밀한 분석이 가능하지만, 태깅 시간이 길어지고 모델 성능이 저하될 수 있다.

따라서 **시스템 목적과 환경에 맞는 분석기를 선택하는 것이 중요하다.**

NLTK (Natural Language Toolkit)

영어를 중심으로 한 자연어 처리 라이브러리. 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등을 제공한다. 영어 외에도 네덜란드어, 프랑스어, 독일어 등 다양한 언어를 지원한다.

- 설치: `pip install nltk`
- 사용 전 필요한 모델을 별도로 다운로드해야 한다.

```
import nltk
nltk.download("punkt") # 토큰화용
nltk.download("averaged_perceptron_tagger") # 품사 태깅용
```

두 모델 모두 **Treebank**라는 대규모 영어 말뭉치 기반으로 학습됐다. Punkt는 통계 기반, Averaged Perceptron Tagger는 퍼셉트론 기반 모델이다.

주요 기능

- **토큰화** — `punkt` 모델 기반으로 동작한다.
 - `word_tokenize()`: 공백 기준으로 단어를 분리하고, 구두점도 별도 토큰으로 추출
 - `sent_tokenize()`: 마침표·느낌표·물음표 등을 기준으로 문장 단위로 분리

```
# 영문 토큰화
from nltk import tokenize

sentence = "Those who can imagine anything, can create the impossible."

word_tokens = tokenize.word_tokenize(sentence)
sent_tokens = tokenize.sent_tokenize(sentence)

print(word_tokens)
print(sent_tokens)
```

▼ 실행 결과

```
['Those', 'who', 'can', 'imagine', 'anything', ',', 'can', 'create', 'the', 'impossible', '.']
['Those who can imagine anything, can create the impossible.']
```

- 품사 태깅 — `averaged_perceptron_tagger` 모델 기반. 먼저 `word_tokenize()` 로 토큰화한 결과를 입력으로 받는다.

```
# 영문 품사 태깅
from nltk import tag
from nltk import tokenize

sentence = "Those who can imagine anything, can create the impossible."

word_tokens = tokenize.word_tokenize(sentence)
pos = tag.pos_tag(word_tokens)

print(pos)
```

표 5.3 Averaged Perceptron Tagger 품사 태그

태그	품사	태그	품사
CC	접속사	PDT	관사
CD	기수(Cardinal Number)	POS	소유격 조사
DT	관형사	PRP	인칭 대명사
EX	there의 대명사형태	PRP\$	소유격 대명사
FW	외래어	RB	부사

태그	품사	태그	품사
JN	전치사	RBR	비교급 부사
JJ	형용사	RBS	최상급 부사
JJR	비교급 형용사	RP	전치사 또는 조동사
JJS	최상급 형용사	VB	기본형 동사
LS	목록 표시	VBD	과거형 동사
MD	조동사	VBG	현재 부사형 동사
NN	단수형 명사	VBN	과거 분사형 동사
NNS	복수형 명사	VBP	1인칭 단수 현재형 동사
NNP	단수형 고유 명사와 서수(Ordinal Number)	VBZ	3인칭 단수 현재형 동사
NNPS	복수형 고유 명사	WDT	관계사
SYM	기호	WP	주격 관계대명사
TO	To 부정사	WP\$	소유격 관계대명사
UH	감탄사	WRB	관계부사

KoNLPy와의 차이

KoNLPy가 한국어 특화 라이브러리라면, NLTK는 **영어 중심의 범용 NLP 라이브러리**

영어는 한국어와 달리 띄어쓰기가 단어 경계와 거의 일치하기 때문에 토큰화가 상대적으로 단순하다.

spaCy

Cython 기반의 오픈소스 NLP 라이브러리. NLTK와 달리 **속도와 정확도**를 최우선으로 하는 머신러닝 기반 라이브러리이다. NLTK가 학습 목적의 다양한 알고리즘과 예제를 제공하는 반면, spaCy는 실제 프로덕션 환경에서의 효율적인 처리에 초점을 맞춘다. 그만큼 NLTK보다 모델이 더 크고 복잡하며 리소스를 더 많이 요구한다.

영어, 한국어, 일본어 등 24개 이상의 언어를 지원하며, GPU 가속도 지원한다.

```
pip install spacy
python -m spacy download en_core_web_sm # 영어 모델 설치
```

사용 방식 — 객체 지향

spaCy는 **객체 지향(Object Oriented)**으로 구현되어 있어 NLTK와 사용 방식이 다르다. 문장을 처리하면 `doc` 객체가 반환되고, 이 안에 여러 `token` 객체가 담긴다.

```
# spaCy 품사 태깅
import spacy

nlp = spacy.load("en_core_web_sm")
sentence = "Those who can imagine anything, can create the impossible."
doc = nlp(sentence)

for token in doc:
    print(f"[{token.pos_:5} - {token.tag_:3}] : {token.text}")
```

▼ 실행 결과

```

[PRON - DT ] : Those
[PRON - WP ] : who
[AUX - MD ] : can
[VERB - VB ] : imagine
[PRON - NN ] : anything
[PUNCT - , ] : ,
[AUX - MD ] : can
[VERB - VB ] : create
[DET - DT ] : the
[ADJ - JJ ] : impossible
[PUNCT - . ] : .

```

표 5.4 spaCy 품사 태그

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
ADJ	JJ	형용사	NUM	CD	기수(Cardinal Number)
ADJ	JJR	비교급 형용사	PROPN	NNP	고유 명사
ADJ	JJS	최상급 형용사	PROPN	NNPS	복수형 고유 명사
ADP	IN	전치사	PUNCT	-LRB-	왼쪽 괄호
ADV	RB	부사	PUNCT	-RRB-	오른쪽 괄호
ADV	RBR	비교급 부사	PUNCT	,	쉼표
ADV	RBS	최상급 부사	PUNCT	:	콜론
ADV	WRB	관계 부사	PUNCT	.	마침표, 물음표, 느낌표
AUX	MD	조동사	PUNCT	...	생략 부호
CCONJ	CC	접속사	PUNCT	"	여는 따옴표
DET	DT	관사, 한정사	PUNCT	"	닫는 따옴표
DET	PDT	전치사 한정사	PUNCT	HYPH	하이픈
DET	PRP\$	소유격 대명사	PUNCT	NFP	불필요한 문장 부호
DET	WDT	관계 대명사	SYM	\$	통화
DET	WP\$	소유격 관계 대명사	VERB	VB	동사 기본형
INTJ	UH	감탄사	VERB	VBD	과거형 동사
NOUN	NN	명사	VERB	VBG	동명사, 현재분사
NOUN	NNS	복수형 명사	VERB	VBN	과거분사
PART	POS	소유격 조사	VERB	VBP	동사 현재형
PART	RP	부사적 불변화사	VERB	VBZ	3인칭 단수 동사 현재형

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
PART	TO	To 부정사	X	ADD	이메일
PRON	EX	존재문(there)	X	FW	외래어
PRON	PRP	인칭 대명사	X	LS	목록
PRON	WP	관계 대명사	X	SYM	기호

NLTK vs. spaCy 비교

	NLTK	spaCy
목적	학습·연구용	실무·프로덕션용
속도	느림	빠름
모델 크기	작음	큼
품사 태그 수	35개	pos_ + tag_ 이중 구조
사용 방식	함수 중심	객체 중심

형태소 분석 & 품사 태깅의 한계와 의의

띄어쓰기·맞춤법 오류, 신조어, 외래어, 축약어 등에는 완벽히 대응하기 어렵다. 그럼에도 형태소 분석과 품사 태깅은 단어의 의미를 파악하고 문장 구조를 이해하는 데 필수적인 전처리 작업으로, 키워드 추출·감성 분석(긍정/부정 판단) 등 언어 모델 성능을 높이는 데 핵심적인 역할을 한다.

하위 단어 토큰화 (Subword Tokenization)

기본 형태소 분석기의 한계

- 형태소 분석기는 모르는 단어를 적절한 단위로 나누는 것에 취약하다. 현대 언어에서는 이 문제가 특히 두드러지는데, 신조어·외래어·오타자·축약어 등이 끊임없이 생겨나기 때문이다.
 - 외래어 문제: '시보리도 짱짱하고 허리도 어병하지안구 죠하효'
→ '시', '보리'로 쪼개짐 ('시보리' 인식 실패)
 - 신조어 문제: '돈꿀날만 하네요'
→ '돈꿀날'로 분해. '돈꿀내다', '돈꿀나다' 등 어근에 붙는 모든 어미 조합이 각각 별개 토큰이 되어 어휘 사전 크기가 폭증하고 OOV 문제가 심화된다.

하위 단어 토큰화란?

- 하나의 단어를 빈번하게 사용되는 하위 단어(Subword)의 조합으로 나누어 토큰화하는 방법. 단어 전체를 하나의 토큰으로 보는 대신, 의미 있는 더 작은 단위로 분리한다.
- 예: 'Reinforcement' → 'Rein', 'force', 'ment'
- 이렇게 하면 처음 보는 단어도 이미 알고 있는 subword 조합으로 처리할 수 있어서, OOV 문제·신조어·은어·고유어 등을 효과적으로 완화할 수 있다. 단어 길이도 줄어들어 처리 속도도 빨라진다.

주요 하위 단어 토큰화 방법

- 바이트 페어 인코딩 (BPE, Byte Pair Encoding)
- 워드피스 (WordPiece)
- 유니그램 모델 (Unigram Model)

바이트 페어 인코딩 (BPE, Byte Pair Encoding)

텍스트에서 가장 빈번하게 등장하는 글자 쌍을 찾아 하나의 단위로 병합하는 하위 단어 토큰화 방법. 원래는 데이터 압축 알고리즘(다이그램 코딩)으로 개발됐으나, NLP에서 하위 단어 토큰화 방법으로 활용된다.

작동 방식

자주 등장하는 글자 쌍을 반복적으로 병합해 어휘 사전을 확장한다. 연속된 글자 쌍이 더 이상 나타나지 않거나, 정해진 어휘 사전 크기에 도달할 때까지 반복한다.

- 'abracadabra' 압축 예시:

```
원문:   abracadabra
Step 1: AracadAra      ('ab' → 'A'로 치환, 빈도 최고)
Step 2: ABCadAB        ('ra' → 'B'로 치환)
Step 3: CcadC          ('AB' → 'C'로 치환)
```

토큰라이저로 활용할 때는 치환 대신 자주 등장하는 글자 쌍을 어휘 사전에 추가하는 방식으로 동작한다.

말뭉치 적용 예시

초기 상태 — 모든 단어를 글자 단위로 분리하고 빈도 사전 구성:

```
빈도 사전: ('l','o','w', 5), ('l','o','w','e','r', 2),
('n','e','w','e','s','t', 6), ('w','i','d','e','s','t', 3)
어휘 사전: ['d','e','i','l','n','o','r','s','t','w']
```

반복 병합:

- 'e'+s' → 'es' 병합 (newest 6번 + widest 3번 = 9번으로 최빈)
- 'es'+t' → 'est' 병합
- 이런 식으로 10번 반복하면:

```
최종 어휘 사전: ['d','e','i','l','n','o','r','s','t','w','es','est','lo','lo
w','ne','new','newest','wi','wid','widest']
```

BPE의 핵심 장점 — OOV 해결

말뭉치에 없던 단어도 기존 어휘 사전의 subword 조합으로 처리할 수 있다.

- 'newer' → 'new' + 'e' + 'r'
- 'wider' → 'wid' + 'e' + 'r'
- 'lowest' → 'low' + 'est'

이처럼 OOV 단어도 이미 학습된 subword 조합으로 토큰화가 가능해 **OOV 문제를 근본적으로 완화**한다. 실제 구현 시에는 대량 말뭉치 학습을 위해 이미 구현된 토큰라이저 라이브러리를 사용한다.

센텐스피스 (Sentencepiece)

구글이 개발한 오픈소스 **하위 단어 토큰라이저 라이브러리**. BPE와 유사한 알고리즘으로 입력 데이터를 토큰화하고 단어 사전을 생성한다. 워드피스, 유니코드 기반 등 다양한 알고리즘을 지원하며, 사용자가 직접 설정할 수 있는 하이퍼파라미터를 제공해 세밀한 토큰라이징이 가능하다.

코포라 (Korpora)

국립국어원이나 AI Hub에서 제공하는 **한국어 말뭉치 데이터를 쉽게 불러올 수 있는 오픈소스 라이브러리**. Python API로 간편하게 사용할 수 있다. 센텐스피스로 토큰라이저를 학습할 때 말뭉치 데이터를 공급하는 역할을 한다.

설치

```
pip install sentencepiece Korpora
```

두 라이브러리는 함께 사용한다. 코포라로 말뭉치를 불러오고, 센텐스피스로 그 말뭉치를 학습해 토큰라이저를 구축하는 흐름이다.

토큰라이저 모델 학습

```
# 청와대 청원 데이터 다운로드
from Korpora import Korpora
```

```
corpus = Korpora.load("korean_petitions")
dataset = corpus.train
petition = dataset[0]

print("청원 시작일 :", petition.begin)
print("청원 종료일 :", petition.end)
print("청원 동의 수 :", petition.num_agree)
print("청원 범주 :", petition.category)
print("청원 제목 :", petition.title)
print("청원 본문 :", petition.text[:30])
```

▼ 실행 결과

Korpora 는 다른 분들이 연구 목적으로 공유해주신 말뭉치들을 손쉽게 다운로드, 사용할 수 있는 기능만을 제공합니다.

말뭉치들을 공유해 주신 분들에게 감사드리며, 각 말뭉치 별 설명과 라이선스를 공유 드립니다. 해당 말뭉치에 대해 자세히 알고 싶으신 분은 아래의 description 을 참고, 해당 말뭉치를 연구/상용의 목적으로 이용하실 때에는 아래의 라이선스를 참고해 주시기 바랍니다.

```
# Description
Author : Hyunjoong Kim lovit@github
Repository : https://github.com/lovit/petitions\_archive
References :
```

청와대 국민청원 게시판의 데이터를 월별로 수집한 것입니다.
청원은 게시판에 글을 올린 뒤, 한달 간 청원이 진행됩니다.
수집되는 데이터는 청원종료가 된 이후의 데이터이며, 청원 내 댓글은 수집되지 않습니다.
단 청원의 동의 개수는 수집됩니다.
자세한 내용은 위의 repository를 참고하세요.

```
# License
CC0 1.0 Universal (CC0 1.0) Public Domain Dedication
Details in https://creativecommons.org/publicdomain/zero/1.0/
```

```
[korean_petitions] download petitions_2017-08: 1.84MB [00:01, 1.33MB/s]
[korean_petitions] download petitions_2017-09: 20.4MB [00:02, 9.32MB/s]
[korean_petitions] download petitions_2017-10: 12.0MB [00:01, 7.10MB/s]
[korean_petitions] download petitions_2017-11: 28.4MB [00:03, 8.55MB/s]
[korean_petitions] download petitions_2017-12: 29.0MB [00:04, 6.34MB/s]
[korean_petitions] download petitions_2018-01: 43.9MB [00:06, 6.97MB/s]
[korean_petitions] download petitions_2018-02: 33.8MB [00:04, 7.26MB/s]
[korean_petitions] download petitions_2018-03: 34.3MB [00:05, 6.73MB/s]
[korean_petitions] download petitions_2018-04: 35.5MB [00:06, 5.35MB/s]
[korean_petitions] download petitions_2018-05: 37.5MB [00:06, 5.67MB/s]
[korean_petitions] download petitions_2018-06: 37.8MB [00:04, 9.39MB/s]
[korean_petitions] download petitions_2018-07: 40.5MB [00:06, 5.91MB/s]
[korean_petitions] download petitions_2018-08: 39.8MB [00:04, 8.30MB/s]
[korean_petitions] download petitions_2018-09: 36.1MB [00:04, 7.23MB/s]
[korean_petitions] download petitions_2018-10: 38.1MB [00:14, 2.64MB/s]
[korean_petitions] download petitions_2018-11: 37.7MB [00:13, 2.71MB/s]
[korean_petitions] download petitions_2018-12: 33.0MB [00:04, 6.81MB/s]
[korean_petitions] download petitions_2019-01: 34.8MB [00:05, 6.73MB/s]
[korean_petitions] download petitions_2019-02: 30.8MB [00:06, 4.98MB/s]
[korean_petitions] download petitions_2019-03: 34.9MB [00:03, 9.22MB/s]
청원 시작일 : 2017-08-25
청원 종료일 : 2017-09-24
청원 동의 수 : 88
청원 범주 : 육아/교육
청원 제목 : 학교는 인력센터, 취업센터가 아닙니다. 정말 간곡히 부탁드립니다.
청원 본문 : 안녕하세요. 현재 시대, 교대 등 교원양성학교들의 예비
```

```
# 학습 데이터셋 생성
from Korpora import Korpora
```

```
corpus = Korpora.load("korean_petitions")
petitions = corpus.get_all_texts()
with open("./datasets/corpus.txt", "w", encoding="utf-8") as f:
    for petition in petitions:
        f.write(petition + "\n")
```

```
# 토큰나이저 모델 학습
from sentencepiece import SentencePieceTrainer

SentencePieceTrainer.Train(
    "--input=./datasets/corpus.txt\
    --model_prefix=petition_bpe\
    --vocab_size=8000 --model_type=bpe"
)
```

▼ 실행 결과

```
sentencepiece_trainer.cc(178) LOG(INFO) Running command: --input=./datasets/corpus.txt --model_prefix=petition
sentencepiece_trainer.cc(78) LOG(INFO) Starts training with :
trainer_spec {
  input: ./datasets/corpus.txt
  input_format:
  model_prefix: petition_bpe
  model_type: BPE
  vocab_size: 8000
  self_test_sample_size: 0
  character_coverage: 0.9995
  input_sentence_size: 0
  shuffle_input_sentence: 1
  seed_sentencepiece_size: 1000000
  shrinking_factor: 0.75
  max_sentence_length: 4192
  num_threads: 16
  num_sub_iterations: 2
  max_sentencepiece_length: 16
  split_by_unicode_script: 1
  split_by_number: 1
  split_by_whitespace: 1
  split_digits: 0
  pretokenization_delimiter:
  treat_whitespace_as_suffix: 0
  allow_whitespace_only_pieces: 0
...
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1625 size=6500 all=1444974 active=83034 piece=적도
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=1625 min_freq=77
trainer_interface.cc(689) LOG(INFO) Saving model: petition_bpe.model
trainer_interface.cc(701) LOG(INFO) Saving vocabs: petition_bpe.vocab
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

▼ SentencePieceTrainer 매개변수

매개변수	의미
input	말뭉치 텍스트 파일의 경로
model_prefix	모델 파일 이름
vocab_size	어휘 사전 크기
character_coverage	말뭉치 내에 존재하는 글자 중 토큰나이저가 다룰 수 있는 글자의 비율

model_type	토크나이저 알고리즘 - unigram - bpe - char - word
max_sentence_length	최대 문장 길이
unk_id	어휘 사전에 없는 OOV를 의미하는 unk 토큰의 id (기본값: 0)
bos_id	문장이 시작되는 지점을 의미하는 bos 토큰의 id (기본값: 1)
eos_id	문장이 끝나는 지점을 의미하는 eos 토큰의 id (기본값: 2)

```
# 바이트 페어 인코딩 토큰화
from sentencepiece import SentencePieceProcessor

tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")

sentence = "안녕하세요, 토크나이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토크나이저를 사용할 수 있답니  
다"]

tokenized_sentence = tokenizer.encode_as_pieces(sentence)
tokenized_sentences = tokenizer.encode_as_pieces(sentences)
print("단일 문장 토큰화 :", tokenized_sentence)
print("여러 문장 토큰화 :", tokenized_sentences)

encoded_sentence = tokenizer.encode_as_ids(sentence)
encoded_sentences = tokenizer.encode_as_ids(sentences)
print("단일 문장 정수 인코딩 :", encoded_sentence)
print("여러 문장 정수 인코딩 :", encoded_sentences)

decode_ids = tokenizer.decode_ids(encoded_sentences)
decode_pieces = tokenizer.decode_pieces(encoded_sentences)
print("정수 인코딩에서 문장 변환 :", decode_ids)
print("하위 단어 토큰에서 문장 변환 :", decode_pieces)
```

▼ 실행 결과

```
단일 문장 토큰화 : ['_안녕하세요', ',', ' ', '_토', '크', '나', '이', '저', '가',
'_잘', '_학', '습', '_되었', '군요', '!']
여러 문장 토큰화 : [['_이렇게', '_입', '력', '값을', '_리', '스트', '로', '_받
아서'], ['_쉽게', '_토', '크', '나', '이', '저', '를', '_사용할', '_수', '_
있', '답니다']]
단일 문장 정수 인코딩 : [667, 6553, 994, 6880, 6544, 6513, 6590, 6523, 161,
110, 6554, 872, 787, 6648]
여러 문장 정수 인코딩 : [[372, 182, 6677, 4433, 1772, 1613, 6527, 4162], [1
681, 994, 6880, 6544, 6513, 6590, 6536, 5852, 19, 5, 2639]]
정수 인코딩에서 문장 변환 : ['이렇게 입력값을 리스트로 받아서', '쉽게 토크나이저를
```

사용할 수 있습니다']

하위 단어 토큰에서 문장 변환 : ['이렇게 입력값을 리스트로 받아서', '쉽게 토큰라이저를 사용할 수 있습니다']

```
# 어휘 사전 불러오기
from sentencepiece import SentencePieceProcessor

tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")

vocab = {idx: tokenizer.id_to_piece(idx) for idx in range(tokenizer.get_piece_size())}
...

get_piece_size : sentencepiece 모델에서 생성된 하위 단어의 개수 반환
id_to_piece : 정수값을 하위 단어로 변환
--> 하위 단어의 개수만큼 반복해 하위 단어 딕셔너리 구성
...

print(list(vocab.items())[:5])
print("vocab size :", len(vocab))
```

▼ 실행 결과

```
[(0, '<unk>'), (1, '<s>'), (2, '</s>'), (3, '니다'), (4, '_이')]
vocab size : 8000
```

워드피스 (Wordpiece)

BPE와의 차이

BPE가 **빈도** 기준으로 글자 쌍을 병합하는 반면, 워드피스는 **확률** 기준으로 병합한다. 단순히 많이 등장한 쌍이 아니라, 병합했을 때 언어 모델의 확률을 가장 높이는 쌍을 선택한다.

글자 쌍 병합 점수 계산:

$$\text{score} = f(x, y) / f(x) \times f(y)$$

- $f(x, y)$: x와 y가 함께 등장한 빈도
- $f(x)$, $f(y)$: x, y 각각의 빈도

점수가 높을수록 두 글자가 독립적으로 등장하는 것보다 **함께 등장하는 경향이 강하다**는 의미이므로 병합 우선순위가 높아진다.

BPE와의 차이 예시:

- 'e' 와 's' 쌍: 9번 등장, 하지만 e는 17번·s는 9번 각각 등장 → $\text{score} = 9/(17 \times 9) \approx 0.06$
- 'i' 와 'd' 쌍: 3번 등장, i와 d 각각 3번 등장 → $\text{score} = 3/(3 \times 3) \approx 0.33$

BPE라면 빈도가 높은 'e'+ 's' 를 병합하지만, 워드피스는 score가 높은 'i'+ 'd' 를 먼저 병합한다.

BPE와 마찬가지로 연속된 글자 쌍이 더 이상 나타나지 않거나, 정해진 어휘 사전 크기에 도달할 때까지 반복한다.

토큰라이저스 (Tokenizers)

허깅페이스(Hugging Face)에서 제공하는 토큰라이저 라이브러리. 워드피스 API를 이용해 쉽고 빠르게 토큰라이저를 구현하고 학습할 수 있다. 센텐스피스 대신 실습에서 활용한다.

```
pip install tokenizers
```

주요 기능

- **정규화 (Normalization)** — 텍스트를 일관된 형식으로 표준화하는 전처리 작업. 모호한 경우를 방지하기 위해 일부 문자를 대체하거나 제거한다.
 - 불필요한 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점 처리, 특수 문자 처리 등
- **사전 토큰화 (Pre-tokenization)** — 본격적인 토큰화 전에 입력 문장을 단어 같은 작은 단위로 먼저 나누는 작업. 공백 또는 구두점을 기준으로 분리해 텍스트를 효율적으로 처리하고 모델 성능을 향상시킨다.

예시 코드

```
# 워드피스 토큰라이저 학습
from tokenizers import import Tokenizer
from tokenizers.models import WordPiece
from tokenizers.normalizers import Sequence, NFD, Lowercase
from tokenizers.pre_tokenizers import Whitespace

tokenizer = Tokenizer(WordPiece())

tokenizer.normalizer = Sequence([NFD(), Lowercase()])
tokenizer.pre_tokenizer = Whitespace()

tokenizer.train(["../datasets/corpus.txt"])
tokenizer.save("../models/petition_wordpiece.json")
```

▼ 정규화 클래스

이름	설명
NFD()	NFD 유니코드 정규화
NFKD()	NFKD 유니코드 정규화
NFC()	NFC 유니코드 정규화
NFKC()	NFKC 유니코드 정규화
Lowercase()	입력 문장의 모든 대문자를 소문자로 변환
Strip()	입력 문장의 양 끝에 있는 공백 제거
StripAccents()	모든 엑센트 기호 제거
Replace()	문자 혹은 정규 표현식을 기반으로 입력 문장 변환
BertNormalizer()	BERT 모델에 사용된 정규화 적용
Sequence()	여러 개의 정규화 모듈을 병합해 순서대로 수행

▼ 사전 토큰화 클래스

클래스	설명
Sequence()	여러 개의 사전 토큰화 모듈을 병합하여 순서대로 수행
ByteLevel()	OpenAI에서 GPT-2를 학습할 때 사용한 방법으로 문자열을 그래픽 문자열로 매핑하고 공백을 기준으로 나눈다.
CharDelimiterSplit()	문자 기준으로 문자열을 나눈다.
Digits()	모든 형태의 숫자 표현을 기준으로 문자열을 나눈다.
Punctuation()	구두점을 기준으로 입력 문자열을 나눈다.
Metaspace(replacement="_ "	Whitespace 기준으로 사전 토큰화를 진행하고, replacement로 Whitespace를 치환한다. ▪ 기본값: _(\u2581)
Whitespace()	단어 경계(공백과 구두점)를 기준으로 사전 토큰화를 진행한다. ▪ <code>\w+!["'\w\s]+</code> 정규 표현식을 적용한다.
WhitespaceSplit()	공백 문자를 기준으로 입력 문자열을 나눈다.
Split(pattern, behavior	정규표현식(pattern)과 동작(behavior)을 기준으로 문자열을 나눈다. ▪ behavior - removed: 삭제 - isolated: 개별 토큰 처리 - merged_with_previous: 이전 문자열과 결합해 토큰 처리 - merged_with_next: 다음 문자열과 결합해 토큰 처리 - contiguous: 다음 문자열이 토큰화되지 않아도 결합해 토큰 처리

```
# 워드피스 토큰화
from tokenizers.decoders import WordPiece as WordPieceDecoder

tokenizer = Tokenizer.from_file("./models/petition_wordpiece.json")
tokenizer.decoder = WordPieceDecoder()

sentence = "안녕하세요, 토큰라이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토큰라이저를 사용할 수 있답니  
다"]

encoded_sentence = tokenizer.encode(sentence)
encoded_sentences = tokenizer.encode_batch(sentences)

print("인코더 형식 :", type(encoded_sentence))

print("단일 문장 토큰화 :", encoded_sentence.tokens)
print("여러 문장 토큰화 :", [enc.tokens for enc in encoded_sentences])

print("단일 문장 정수 인코딩 :", encoded_sentence.ids)
print("여러 문장 정수 인코딩 :", [enc.ids for enc in encoded_sentences])

print("정수 인코딩에서 문장 변환 :", tokenizer.decode(encoded_sentence.ids))
```

▼ 실행 결과

```
인코더 형식 : <class 'tokenizers.Encoding'>
단일 문장 토큰화 : ['안녕하세요', ',', '토', '##크', '##나이', '##저', '##가',  
'잘', '학습', '##되었', '##군요', '!']
여러 문장 토큰화 : [['이렇게', '입력', '##값을', '리스트', '##로', '받아서'],
```

```
['쉽게', '토', '##크', '##나이', '##저', '##를', '사용할', '수', '있다', '##  
됩니다']]
```

단일 문장 정수 인코딩 : [8760, 11, 8693, 8415, 16269, 7536, 7488, 7842, 15016, 8670, 8734, 0]

여러 문장 정수 인코딩 : [[8187, 19643, 13834, 28119, 7495, 12607], [9739, 8693, 8415, 16269, 7536, 7510, 14129, 7562, 8157, 7489]]

정수 인코딩에서 문장 변환 : 안녕하세요, 토큰라이저가 잘 학습되었군요!

6. 임베딩 (~TF-IDF)

언어 모델 (Language Model)

자기회귀 언어 모델 (Autoregressive Language Model)

통계적 언어 모델 (Statistical Language Model)

N-gram

TF-IDF (Term Frequency-Inverse Document Frequency)

단어 빈도 (Term Frequency, TF)

문서 빈도 (Document Frequency, DF)

역문서 빈도 (Inverse Document Frequency, IDF)

TF-IDF

텍스트 벡터화 (Text Vectorization)

- 왜 필요한가?

토큰화만으로는 모델을 학습할 수 없다. 컴퓨터는 텍스트 자체를 이해하지 못하기 때문에 **텍스트를 숫자로 변환**하는 과정이 필요하다. 이를 텍스트 벡터화라고 한다.

- One-Hot Encoding

각 단어를 고유한 색인 값으로 매핑한 뒤, 해당 색인 위치만 1, 나머지는 모두 0으로 표현하는 방식.

예: 'I like apples', 'I like bananas'

색인	0	1	2	3
토큰	I	like	apples	bananas

I like apples → [1, 1, 1, 0]
I like bananas → [1, 1, 0, 1]

- 빈도 벡터화(Count Vectorization)

단어의 등장 빈도를 세어 벡터로 표현하는 방식. 원-핫 인코딩이 등장 여부(0/1)만 표현하는 반면, 빈도 벡터화는 **실제 등장 횟수**로 표현한다. **apples** 가 4번 등장하면 벡터값이 4가 된다.

- 두 방식의 공통 한계

- 희소성(Sparsity) 문제** — 말뭉치의 토큰 수만큼 벡터 차원이 필요하지만, 실제 문장에서 사용되는 토큰은 훨씬 적어 대부분의 값이 0이 된다. 차원의 저주와 컴퓨팅 비용 증가로 이어진다.
- 의미 미반영 문제** — 벡터가 텍스트의 의미를 담지 않는다. 'i scream for ice cream' 과 'ice cream for i scream' 은 의미가 전혀 다르지만 둘 다 [1, 1, 1, 1, 1] 로 동일하게 표현된다.

- 한계를 극복하는 방법 — 임베딩

- 워드 임베딩 (Word Embedding)** — 단어를 고정된 길이의 실수 벡터로 표현. 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간 관계를 추론할 수 있다. Word2Vec, fastText 등이 대표적이다.
- 단, 워드 임베딩은 고정된 임베딩을 학습하기 때문에 **다의어나 문맥 정보를 다루기 어렵다**는 한계가 있다. 이를 해결하기 위해 인공 신경망을 활용한 **동적 임베딩(Dynamic Embedding)** 기법을 사용한다.

언어 모델 (Language Model)

$P(\text{대한민국 반갑습니다}) = 0.081$	$P(\text{서울특별시}) = 0.59$
Input : 안녕하세요 ...	Input : 대한민국 강남구 ...
$P(\text{아이디오 괜찮습니다}) = 0.000001$	$P(\text{아이디스 아메리카노}) = 0.000001$

그림 6.1 언어 모델

입력된 문장을 바탕으로 **다음에 올 문장이나 단어의 확률을 계산**하는 모델. 문맥을 이해하고 문장 구성을 예측한다. 자동 번역, 음성 인식, 텍스트 요약 등 다양한 NLP 분야에서 활용된다.

완성된 문장 단위로 확률을 계산하는 것은 경우의 수가 무한에 가깝기 때문에 사실상 불가능하다. 그래서 **토큰 단위로 하나씩 예측**하는 방식인 자기회귀 언어 모델이 고안됐다.

자기회귀 언어 모델 (Autoregressive Language Model)

이전에 등장한 모든 토큰의 정보를 바탕으로 **다음에 올 토큰을 하나씩 예측**하는 모델. 모델의 출력값이 다시 입력값으로 사용되기 때문에 **자기회귀**라는 이름이 붙었다. 토큰 분류 문제로 정의할 수 있다.

조건부 확률과 연쇄법칙

수식 6.1 언어 모델의 조건부 확률

$$P(w_i | w_1, w_2, \dots, w_{i-1}) = \frac{P(w_1, w_2, \dots, w_i)}{P(w_1, w_2, \dots, w_{i-1})}$$

수식 6.2 조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

$$P(w_i | w_1, w_2, \dots, w_{i-1}) = P(w_1)P(w_2 | w_1) \dots P(w_i | w_1, w_2, \dots, w_{i-1})$$

- 언어 모델은 조건부 확률의 연쇄법칙으로 문장 전체의 확률을 계산한다.
- 즉, 문장 전체의 확률 = 첫 번째 단어의 확률 × 각 단어가 이전 단어들을 조건으로 등장할 확률의 곱

동작 흐름

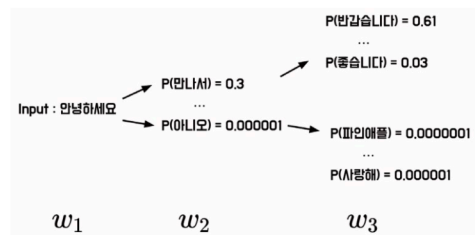


그림 6.2 조건부 확률의 연쇄법칙을 통한 자기회귀 언어 모델 구조

- 각 시점에서 이전 토큰 전체를 조건으로 삼아 다음 토큰을 예측하고, 그 출력이 다시 입력이 되어 예측이 이어진다. 이를 통해 언어 모델은 문장 전체의 확률을 계산하고 다음 단어를 생성한다.

통계적 언어 모델 (Statistical Language Model)

언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성·분석하는 모델. 시퀀스에 대한 확률 분포를 추정해 다음에 등장할 단어의 확률을 예측한다. 일반적으로 **마르코프 체인(Markov Chain)**을 이용해 구현되며, 이전 상태와 현재 상태 간의 전이 확률로 다음 상태를 예측한다.

빈도 기반 조건부 확률

- 말뭉치에서 각 단어가 등장한 빈도수를 기반으로 확률을 계산한다.
- 예: 말뭉치에서 '안녕하세요' 1000번, '안녕하세요 만나서' 700번, '안녕하세요 반갑습니다' 100번 등장 시

수식 6.3 빈도 기반의 조건부 확률

$$P(\text{만나서} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

한계 — 데이터 희소성 (Data Sparsity)

- 단어의 순서와 빈도에만 기초하므로 문맥을 제대로 파악하지 못할 수 있다. 또한 **한 번도 등장한 적 없는 단어나 문장에 대해서는 정확한 확률을 예측하기 어렵다**. 이처럼 관측한 적 없는 데이터를 예측하지 못하는 문제를 데이터 희소성(Data Sparsity) 문제라고 한다.

현대적 활용 — GPT, BERT

- 통계적 언어 모델은 기존 학습 데이터에서 패턴을 찾아 확률 분포를 생성하므로 새로운 문장 생성이 가능하고, 딥러닝 기술과 결합해 더욱 강력한 모델로 발전했다. 최근의 언어 모델은 가중치를 **사전 학습(pre-training)** 하는 방식을 사용한다.
- GPT** (Generative Pre-trained Transformer) — 앞선 토큰들을 보고 다음 토큰을 순차적으로 예측하는 **인과적(causal)** 언어 모델

Input: 'GPT는 OpenAI에서 개발한'
→ Prediction-1: 인과적
→ Prediction-2: 언어 모델입니다.

- BERT** (Bidirectional Encoder Representations from Transformers) — 문장 내 일부 토큰을 [MASK]로 가리고 이를 예측하는 **마스킹** 방식

Input: 'BERT는 [MASK]에서 개발한 [MASK] 언어 모델입니다.'
→ Prediction: Google, 마스킹된

- GPT와 BERT 모두 신경망을 통해 확률 P를 계산할 수도 있고, 통계적 방법으로도 계산할 수 있다.

N-gram

텍스트에서 **N개의 연속된 단어 시퀀스**를 하나의 단위로 취급해 특정 단어 시퀀스가 등장할 확률을 추정하는 가장 기초적인 통계적 언어 모델.

N값에 따라 명칭이 달라진다:

- N=1 → **유니그램(Unigram)**: 단어 하나씩 독립적으로
- N=2 → **바이그램(Bigram)**: 연속된 2개 단어씩
- N=3 → **트라이그램(Trigram)**: 연속된 3개 단어씩
- N≥4 → N-gram

예: '안녕하세요 만나서 진심으로 반가워요'

Unigram : <u>안녕하세요</u> <u>만나서</u> <u>진심으로</u> <u>반가워요</u>	<u>안녕하세요</u> , <u>만나서</u> , <u>진심으로</u> , <u>반가워요</u>
Bigram : <u>안녕하세요 만나서</u> <u>진심으로 반가워요</u>	<u>안녕하세요 만나서</u> , <u>만나서 진심으로</u> , <u>진심으로 반가워요</u>
Trigram : <u>안녕하세요 만나서 진심으로</u> <u>반가워요</u>	<u>안녕하세요 만나서 진심으로</u> , <u>만나서 진심으로 반가워요</u>

그림 6.3 N이 1, 2, 3인 N-gram

조건부 확률 계산 방식

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

- 모든 이전 토큰을 고려하지 않고 N-1개의 토큰만 고려해 확률을 계산한다.
- N값을 조정해 모델의 성능을 조절할 수 있다.

구현

```
# N-gram
import nltk

def ngrams(sentence, n):
    words = sentence.split()
    ngrams = zip(*[words[i:] for i in range(n)])
    return list(ngrams)

sentence = "안녕하세요 만나서 진심으로 반가워요"

unigram = ngrams(sentence, 1)
bigram = ngrams(sentence, 2)
trigram = ngrams(sentence, 3)

print(unigram)
print(bigram)
print(trigram)

# NLTK 라이브러리로도 동일하게 구현 가능
unigram = nltk.ngrams(sentence.split(), 1)
bigram = nltk.ngrams(sentence.split(), 2)
trigram = nltk.ngrams(sentence.split(), 3)

print(list(unigram))
print(list(bigram))
print(list(trigram))
```

▼ 실행 결과

```
[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]
[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]
```

활용 및 특징

- 작은 규모의 데이터셋에서 **연속된 문자열 패턴 분석**에 효과적이며, **관용적 표현 분석**에도 활용된다. 예를 들어 '입이 무겁다' 처럼 자주 등장하는 연속 단어를 추출해 관용 표현을 파악할 수 있다.
- 단어의 순서가 중요한 NLP 작업 및 문자열 패턴 분석에 적합하지만, 통계적 언어 모델의 공통 한계인 **데이터 희소성 문제**는 여전히 존재한다.

TF-IDF (Term Frequency-Inverse Document Frequency)

텍스트 문서에서 **특정 단어의 중요도를 계산**하는 통계적 가중치. BoW에 가중치를 부여하는 방법이다.

BoW(Bag-of-Words) 는 문서를 단어의 집합으로 표현하며 등장 빈도를 기록한다. 원-핫 인코딩이 0/1로만 표현하는 것과 달리 BoW는 빈도수를 그대로 반영한다.

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

그런데 BoW는 모든 단어에 동일한 가중치를 부여하기 때문에 문제가 생긴다. 'I like this movie' 와 'I don't like this movie' 는 'don't' 하나로 의미가 완전히 달라지지만, 'don't' 는 자주 등장하지 않아 분류 시 무시될 수 있다. 이를 해결하기 위해 TF-IDF를 사용한다.

단어 빈도 (Term Frequency, TF)

문서 내에서 특정 단어가 등장한 횟수.

$$TF(t, d) = count(t, d)$$

- TF가 높을수록 해당 문서에서 자주 쓰이는 단어 → 전문 용어나 관용어일 가능성
- 단, 문서가 길수록 TF 값도 자연히 높아지는 한계가 있음

문서 빈도 (Document Frequency, DF)

한 단어가 **몇 개의 문서**에 등장하는지를 나타내는 값.

$$DF(t, D) = count(t \in d : d \in D)$$

예: 3개 문서에서 'movie' 가 총 4번 등장해도, 3개 문서 모두에 있으면 DF = 3

- DF 높음 → 많은 문서에서 공통적으로 쓰임 → **일반적인 단어, 중요도 낮음**
- DF 낮음 → 특정 문서에서만 등장 → **특수한 맥락의 단어, 중요도 높음**

역문서 빈도 (Inverse Document Frequency, IDF)

DF의 역수에 로그를 취한 값. **단어가 희귀할수록 IDF가 커진다.**

$$IDF(t, D) = \log\left(\frac{count(D)}{1+DF(t, D)}\right)$$

- 분모에 1을 더하는 이유: 한 번도 등장하지 않은 단어의 경우 분모가 0이 되는 것을 방지
- 로그를 취하는 이유: 전체 문서 수 대비 DF가 매우 작으면 IDF가 지나치게 커질 수 있기 때문 (예: 10,000개 문서에서 1번 등장 시 IDF = 5,000 → 로그로 스케일 조정)

TF-IDF

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

특정 문서에서 자주 등장(TF↑)하면서 전체 문서에서는 드물게 등장(IDF↑)할수록 TF-IDF 값이 커진다. 반대로 관사나 관용어처럼 전체 문서에서 자주 등장하는 단어는 가중치가 낮아진다.

TD-IDF Vectorizer 주요 매개변수

```
tfidf_vectorizer = sklearn.feature_extraction.text.TfidfVectorizer(  
    input="content",          # 입력 형태: content(문자열), file(파일 객체), filename(파일 경로)  
    encoding="utf-8",         # 텍스트 인코딩  
    lowercase=True,          # 소문자 변환 여부  
    stop_words=None,          # 불용어 지정 (지정 시 단어 사전에서 제외)  
    ngram_range=(1, 1),      # N-gram 범위. (1,1)=유니그램, (1,2)=유니그램+바이그램  
    max_df=1.0,               # 이 비율/횟수 초과 등장 단어는 불용어 처리  
    min_df=1,                 # 이 비율/횟수 미만 등장 단어는 불용어 처리  
    vocabulary=None,          # 미리 구축한 단어 사전 사용. None이면 자동 구축
```

```
smooth_idf=True,      # IDF 분모에 1을 더해 0 나누기 방지
)
```

구현 코드

```
# TF-IDF 계산
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "That movie is famous movie",
    "I like that actor",
    "I don't like that actor"
]

tfidf_vectorizer = TfidfVectorizer()
tfidf_vectorizer.fit(corpus)
tfidf_matrix = tfidf_vectorizer.transform(corpus)

print(tfidf_matrix.toarray())
print(tfidf_vectorizer.vocabulary_)
```

▼ 실행 결과

```
[[0.          0.          0.39687454 0.39687454 0.          0.79374908
  0.2344005 ]
 [0.61980538 0.          0.          0.          0.61980538 0.
  0.48133417]
 [0.4804584  0.63174505 0.          0.          0.4804584  0.
  0.37311881]]
{'that': 6, 'movie': 5, 'is': 3, 'famous': 2, 'like': 4, 'actor': 0, 'don': 1}
```

TF-IDF의 한계

TF-IDF 값이 가장 높은 단어를 문서별로 추출하면 핵심 단어를 파악할 수 있다. 하지만 두 가지 근본적인 한계가 있다.

- **순서·문맥 무시** — 빈도 기반이라 단어의 순서나 문맥을 반영하지 않는다. 문장 생성처럼 순서가 중요한 작업에는 부적합하다.
- **의미 미반영** — 벡터가 문서 내 중요도를 나타낼 뿐, 단어의 의미 자체를 담지 않는다. '나는 바나나를 먹었다'와 '그녀가 과일을 섭취한다'는 의미적으로 가깝지만 벡터 유사도는 '고양이는 야옹 운다'와 동일하게 계산된다.